CAIP: Detecting Router Misconfigurations with Context-Aware Iterative Prompting of LLMs

Paper #722, 13 pages body, 15 pages total

Abstract

Model checkers and consistency checkers detect critical errors in router configurations, but these tools require significant manual effort to develop and maintain. LLM-based Q&A models have emerged as a promising alternative, allowing users to query partitions of configurations through prompts and receive answers based on learned patterns, thanks to transformer models pre-trained on vast datasets that provide generic configuration context for interpreting router configurations. Yet, current methods of partition-based prompting often do not provide enough network-specific context from the actual configurations to enable accurate inference. We introduce a Context-Aware Iterative Prompting (CAIP) framework that automates network-specific context extraction and optimizes LLM prompts for more precise router misconfiguration detection. CAIP addresses three challenges: (1) efficiently mining relevant context from complex configuration files, (2) accurately distinguishing between pre-defined and user-defined parameter values to prevent irrelevant context from being introduced, and (3) managing prompt context overload with iterative, guided interactions with the model. Our evaluations on synthetic and real-world configurations show that CAIP improves misconfiguration detection accuracy by more than 30% compared to partition-based LLM approaches, model checkers, and consistency checkers, uncovering over 20 previously undetected misconfigurations in real-world configurations.

1 Introduction

Ensuring the stability, security, and performance of network infrastructures hinges on careful management when introducing a new routing device or modifying the configuration of an existing device. Even a single device's misconfiguration can lead to black holes, unintended network access, inefficient routes, and a range of other performance and security vulnerabilities. For instance, a misconfigured access control list (ACL) may unintentionally allow unauthorized traffic, exposing the device (and potentially the broader network) to malicious attacks; a buggy routing policy may create loops, thereby degrading performance and rendering local services inaccessible.

The most accurate but costly approach to verifying network configuration updates is to involve a domain expert, such as a network engineer. These experts bring an extensive and comprehensive set of background knowledge gathered over years of experience, vast documentation, and an understanding of standard or best practices in routing configurations. Given a configuration under review, the expert iterates through it, identifies issues, and formulates these issues into policies that future model checkers can use to detect similar misconfigurations. However, this manual process is labor-intensive, costly, and hard to scale, especially in large and dynamic network environments.

To address these limitations, researchers and practitioners have developed automated tools such as model checkers and consistency checkers. - **Model checkers** [1, 3, 4, 14, 18, 36, 39, 43, 54, 57] rely on manually defined *policies*, which include correctness constraints and protocol-specific requirements that configurations must satisfy, enabling them to detect complex misconfigurations accurately. However, curating these policies is challenging. Tailoring rules to different scenarios, device types, software versions, and vendor-specific features requires significant effort, and creating a comprehensive set of policies is often infeasible. Unlike human experts, model checkers cannot relate disparate pieces of information within a configuration without explicit rules. - **Consistency checkers** [13, 19, 20, 22–24, 45], on the other hand, automate the process by comparing configurations across devices or best-practice baselines. These tools flag deviations as potential issues. While this approach requires minimal manual intervention, it lacks semantic depth. Deviations do not necessarily imply faults, and consistency-based tools cannot leverage domain knowledge or policies derived from expert insights.

Given these challenges, recent efforts start to explore the feasibility of using large language models (LLMs) as domain experts to carry out the task of misconfiguration detection. LLMs are pre-trained on extensive corpora that encompass network-related texts, including protocol specifications,

vendor documentation, and publicly available configurations. This makes them analogous to human experts with broad domain knowledge. By feeding the configuration under review to an LLM, the model can iterate through it and identify issues, potentially offering a scalable and automated alternative to manual inspection.

Existing LLM-based tools [7, 11, 29, 32, 50, 51] attempt to achieve this by passing entire or partitioned configurations to the LLM in prompts. However, this approach has limitations. Like human engineers, LLMs require *context* to identify issues. Human experts do not examine configurations in a rigid, top-down fashion; instead, they identify relevant configuration lines and dynamically "jump" to related parts of the configuration, regardless of partition boundaries. Similarly, LLMs perform better when provided with concise, relevant context that reflects these interactions. Feeding the entire configuration at once risks exceeding token limits, and splitting configurations into arbitrary partitions can obscure dependencies between different parts. Furthermore, overloading the model with irrelevant or excessive information can dilute its reasoning capabilities, much like a human struggling to recall long-past details.

In this paper, we propose a framework for efficiently extracting and feeding context into LLMs to address these issues and improve misconfiguration detection in routing devices. Specifically, we introduce the *Context-Aware Iterative Prompting* (CAIP) framework, which tackles three key challenges:

Challenge 1: Automatic and Accurate Context Mining: Router configurations are long and complex [5], featuring interdependent lines. Imagine a domain expert tasked with analyzing a configuration file. Instead of reviewing every line exhaustively, they focus on key details and related information. Similarly, automatically extracting and selecting relevant context is essential to reduce computational expense and improve detection accuracy in LLMs. **Solution:** We model the configuration as a hierarchical tree, where each line corresponds to a path from the root to a parameter value. This structure enables systematic identification of three main categories of context—*neighboring*, *similar*, and *referenced* configuration lines—ensuring the LLM receives a concise yet sufficient corpus for analysis.

Challenge 2: Parameter-Value Ambiguity in Referenceable Context: Configurations contain pre-defined and user-defined parameters, with the latter often requiring specialized context for accurate interpretation. Consider a scenario where an expert encounters a user-defined parameter (*e.g.*, an ACL name) and must track its references across the configuration. Similarly, LLMs require clarity in resolving such references. Mislabeling or omitting these references inflates computational costs and increases error risk. **Solution:** We employ an existence-based labeling approach within the configuration tree to distinguish user-defined values by detecting their occurrences as internal nodes in other paths. These classifications are refined with a majority-voting scheme to ensure consistency across contexts.

Challenge 3: Managing Context Overload in Prompts: When faced with an overwhelming volume of irrelevant information, even the best experts can lose focus. Likewise, even after extracting relevant information, the quantity of required context can exceed a single prompt's capacity. Overloading an LLM with excessive detail reduces coherence and degrades performance. **Solution:** In our iterative prompting framework, the LLM explicitly requests further context when needed, incrementally refining the prompt and focusing on the most critical lines. This approach mitigates overload and enables more precise detection.

We evaluate CAIP through two distinct case studies. First, we analyze real-world device configuration snapshots, inject realistic updates (including both correct modifications and latent misconfigurations), and demonstrate that CAIP outperforms partition-based LLM approaches, model checkers, and consistency checkers by at least 30% in error detection accuracy. Second, we exhaustively evaluate CAIP on configuration snapshots from a medium-sized campus network. CAIP identifies over 20 previously overlooked misconfigurations, highlighting its practical utility.

2 Background and Motivation

Network misconfigurations are a common and persistent issue, often leading to security vulnerabilities, performance degradation, or network outages [13, 56]. Manually identifying and rectifying misconfigurations is challenging due to the complexity and interdependency of configurations [5, 25].

Existing methods for automated network misconfiguration detection fall into two categories: model checking and consistency checking. Model checkers [1,3,4,14,18,36,39,43,54,57] identify misconfigurations by deriving a logical model of the control and data planes and executing or analyzing the model to determine whether engineer-specified forwarding policies are satisfied. For example, Batfish simulates the control and data planes under specific network conditions (e.g., link failures) and checks whether forwarding paths deviate from specified policies [14]. Similarly, Minesweeper uses a satisfiability solver to verify whether reachability and load-balancing policies are satisfied under a range of network conditions (e.g., all single link failures) [4]. Although model checkers are popular, they rely on predefined rules/policies and require domain expertise to set up. Moreover, they often struggle with the nuances of particular protocols and vendors [6, 54]. In contrast to model checkers, consistency checkers such as *Diffy* [20] and *Minerals* [23] take a statistical approach by learning common configuration patterns and detecting deviations as potential errors. For instance, Diffy learns configuration templates and identifies anomalies by comparing new configurations to learned patterns, while Minerals applies association rule mining to detect misconfigurations by learning local policies from router configuration files. Unfortunately,

these approaches tend to oversimplify configurations by treating deviations from standard patterns as misconfigurations, leading to false positives when valid configurations deviate for context-specific reasons.

In light of the shortcomings of these existing tools, researchers have looked to Large Language Models (LLMs) for misconfiguration detection [7, 11, 32, 50, 51] due to their advanced ability to understand and process complex contextual information embedded within network configurations. In a Q&A format, prompts containing instructions and configuration lines are provided to the models, which then respond to identify potential misconfigurations. A representative tool in this category is Ciri [29], a LLM-based configuration verifier.

LLMs, particularly transformer-based models [17, 30, 49], excel at capturing intricate relationships between configuration elements by leveraging self-attention mechanisms that dynamically weigh the importance of each token (or configuration parameter) in relation to others, regardless of their distance within the file. This allows LLMs to incorporate both local and global dependencies, enabling them to recognize not only syntax and pattern anomalies but also to infer potential misconfigurations based on the underlying semantics of the configuration. The use of position encodings ensures that the order of elements in a configuration file is considered, allowing LLMs to assess the correctness of parameters based on their sequence. Additionally, most LLMs rely on large-scale pre-trained models (PTMs) trained on large amounts of data [38], particularly generative pretrained tranformers (GPTs) [2, 8, 12, 40, 46, 48]. These mechanisms enable them to obtain vast pre-learned contexts which allow for effective generalization across various tasks. These distinct advantages have spurred the use of LLMs across diverse domains [9, 10, 16, 33, 35, 41], making their application in network misconfiguration detection a natural progression.

Through this architecture and the pre-learned general network configuration contexts, LLM-based Q&A models detect subtle errors that might arise from interactions between different configuration elements—issues often missed by conventional tools. As networks become more complex, with increasingly interdependent components, LLMs' ability to holistically analyze configurations and understand the intent behind them makes them a powerful tool for enhancing the accuracy and reliability of misconfiguration detection. However, to fully leverage this capability, query prompts must include all relevant context from the configuration file when analyzing a particular configuration line or excerpt, including related lines that might aid in detecting misconfigurations. Doing so helps to refine the model's response by incorporating network-specific context unique to the current configuration, rather than relying solely on pre-learned knowledge. Without this targeted context, the LLM's ability to identify potential issues is significantly diminished [21, 31, 42, 47].

Limitations of Full-File Prompting: A naïve approach to incorporating context would be to feed the entire configuration file to the LLM, either all at once in a single prompt or progressively, and let the model handle the detection. This method, while straightforward, is highly inefficient due to the inherent token length limitations of LLMs [15, 53, 55]. More critically, flooding the model with all the configuration data can lead to context overload [26, 28, 37], a known issue where the presence of excessive and irrelevant information dilutes the LLM's capacity to focus on important aspects of the configuration. Context overload not only impairs the model's performance but can also result in a failure to detect important misconfigurations, or generating false positives due to irrelevant context.

Challenges of Partition-Based Prompting: A common solution to context overload has been partition-based prompting, where the large and complex configuration file is broken down into smaller chunks or sections, typically based on the order in which the configuration appears in the file. These chunks are then individually fed to the LLM for misconfiguration detection as independent prompts. This approach reduces the token load per prompt and ensures that the model is not overwhelmed by a massive context. However, this method introduces a new set of problems: by treating sections in isolation, it often fails to account for interdependent lines that reside in different parts of the file. Network configurations, unlike typical documents, often contain parameters that interact with or depend on other sections that may not be adjacent in the file. While methods like prompt chaining [7, 51] guide LLMs by linking subsequent instructions across prompts, these do not target context chaining, leaving critical interdependencies unaccounted for.

For example, consider a configuration file where one chunk defines firewall rules and another chunk, further down the file, defines routing policies. A misconfiguration in the firewall rules might depend on how the routing policies are established, but because partition-based prompting processes these sections independently, this critical context is lost. Lost context is particularly detrimental for detecting dependency-related misconfigurations, where proper detection requires understanding how different sections of the configuration work together. If the partitioned chunk only contains locally adjacent lines that define completely different things, then critical context from other parts of the configuration will be missing, and the model will fail to make accurate inferences.

In this paper, we present a tool that addresses these challenges through a context-aware, iterative prompting mechanism. This approach efficiently manages and extracts relevant context for each configuration line under review, ensuring the LLMs receive the necessary information in prompts without suffering context overload. The next section outlines the challenges we faced in developing this solution.

3 Method

To address the limitations of partition-based prompting, which often fails to capture dependencies across non-adjacent

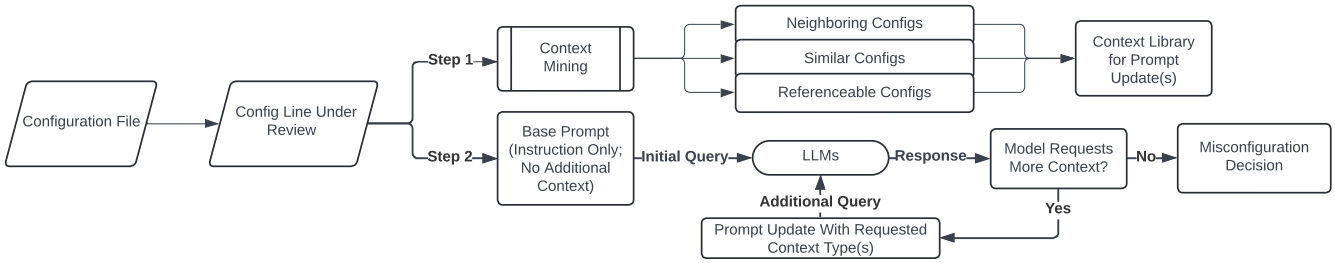

Figure 1: CAIP system overview.

sections of configuration files, we design Context-Aware Iterative Prompting (CAIP), which has two components, as shown in Figure 1: a context mining component and an iterative prompting component. These components work together to automate the process of context extraction and prompt interaction with LLMs, enabling more accurate router misconfiguration detection.

1. Context Mining Component: In this phase, CAIP mines all relevant context for a given configuration line under examination. Relevance here refers to configurations that, while not necessarily directly related, provide important insights, such as neighboring configurations, similar lines applied in different contexts, or referenceable configurations that define key parameters.

2. Iterative Prompting Component: Once the relevant context has been mined, the online component engages the LLM through an iterative prompting process. Instead of overwhelming the model with all the extracted context at once, CAIP interacts with the LLM in a guided, sequential manner.

We now explain the challenges encountered in designing these two components and present the solutions.

3.1 Context Mining

3.1.1 Challenge 1 - Efficient and accurate context mining

Configuration files are often lengthy and complex, consisting of multiple related and interdependent lines that define various aspects of the behavior of the corresponding network device. When analyzing a specific configuration line, efficiently identifying extracting the relevant context can reduce the computational burden on LLMs and minimize the risk of introducing irrelevant information that could degrade the accuracy of misconfiguration detection. This task is challenging because, although these configurations are typically written in a machine and human-readable format, automating the process of context extraction requires a deep understanding of the hierarchical and interconnected nature of the configuration data. For example, in a network configuration, a line that specifies an access control rule might depend on prior lines that define network segments or user roles. Without properly extracting and including this context in prompt, the LLM might misinterpret the rule, leading to false positives or missed misconfigurations.

```
Root (Config)
├─version
│   └─"17.3R3-S7.2"
├─groups
│   ├─INTERFACE-BACKBONE
│   │   └─interfaces
│   │       └─<ge-*>
│   │           └─mtu = "9192"
│   │               └─unit <*>
│   │                   ├─family inet
│   │                   │   └─mtu="9174"
│   │                   │   └─filter
│   │                   │       ├─input-list[0]="inet-sample"
│   │                   │       ├─input-list[1]="backbone-in"
│   │                   │       └─output = "interface-out"
│   │                   └─family mpls
│   │                       └─mtu="9174"
│   │                       └─filter
│   │                           └─input-list="mpls-sample"
│   /* intermediate configurations not shown here
│   ├─firewall
│   │   └─family inet
│   │       └─filter inet-sample
│   │           └─term default
│   │               ├─then[0]="sample"
│   │               └─then[1]="next term"
```

Figure 2: Example snippet of tree-formatted Junpiter router configuration file.

To address this challenge, we rely on two observations:

1. Hierarchical Structure of Configuration Files: Configuration files are typically written in a structured, hierarchical format that can be effectively modeled as a tree, as shown in the example in Figure 2. In this tree representation (T), each node (V) corresponds to a specific configuration element, and edges represent the relationships or dependencies between these elements. Let any unique path $P = \{V_1 \rightarrow V_2(P) \rightarrow V_3(P) \rightarrow \ldots \rightarrow V_k(P) = v_k(P)\}$ in this tree have k nodes, where the value of k can vary depend on the path taken. The root node (V_1) represents the entry point of the configuration file (thus, it functions solely as the root of the tree, providing no semantic meaning, and remains identical regardless of the path taken), while the immediate following node ($V_2(P)$) corresponds to the broadest configuration category in this specific path. Intermediate nodes at subsequent levels ($V_2(P), V_3(P), \ldots, V_{k-1}(P)$) represent increasingly nested configuration sections or subcategories, each refining the configuration context. The leaf node $V_k(P)$ (or parameter node) represents the final configurable parameter in this path, with the associated value $v_k(P)$, *i.e.*, parameter values, indicating the specific configuration setting. An unique configuration line is thus a complete path in the tree, starting from the root node and terminating at a parameter node, where the path reflects the hierarchical structure and relationships defined in the configuration file.

2. Contextual Relevance from Different Aspects: We observe that the context related to any given configuration line falls into one of three types, based on its position and relation within the configuration file as shown in the example in Figure 3, with increasing levels of complexity to mine. They each providing unique insights that contribute to accurate misconfiguration detection:

Neighboring Configurations are closely related to the line under examination, typically within the same section or functional block of the network configuration. For instance, if analyzing a line that defines a VLAN assignment on a particular interface, neighboring configurations would include other lines that configure the same interface or VLAN. These configurations provide insights into how related parameters are set up in proximity, revealing potential dependencies or conflicts. Additionally, by referencing related configurations that define or modify these neighboring parameters, one can understand how changes in one part of the network might impact adjacent configurations, helping to identify misconfigurations that could lead to issues like traffic misrouting or security vulnerabilities. Neighboring configurations are relatively easy to mine because they are identified based on adjacency or location in the configuration file, making them straightforward to extract.

Similar Configurations involve the same type of parameter or function as the configuration line under review but are applied in different contexts within the network. For example, consider configurations that assign IP addresses to different interfaces across various routers in the network. Even though the interfaces and routers may differ, the principles governing IP address assignment remain the same. By comparing these similar configurations, one can detect inconsistencies or deviations from standard practices that might indicate a misconfiguration. This type of context is essential for ensuring that configuration practices are consistent across the network, reducing the risk of errors that could lead to network outages or performance degradation. Mining these configurations is more challenging because, unlike neighboring configurations, they are not located adjacently but must be identified based on functional similarities.

Referenceable Configurations provide essential definitions or additional information related to the parameter value being examined. In the network domain, this context is important for understanding how specific values are applied across different parts of the network configuration. For example, if a parameter value specifies an import policy, referenceable configurations might include other lines where this policy is defined or where its behavior is modified. By examining these configurations, one can gain a deeper understanding of how the policy influences routing decisions or interacts with other network elements, ensuring that the configuration is applied correctly and consistently throughout the network. Referenceable configurations are the most difficult to mine because they often involve indirect references and dependencies

```
Example Configuration Line Under Review for Misconfiguration Detection
groups → INTERFACE-BACKBONE → interfaces → <ge-*> → unit <*>
→ family inet → filter → input-list = "inet-sample"

Example Extracted Contexts:

Neighboring Configurations
(a) groups → INTERFACE-BACKBONE → interfaces → <ge-*> → unit <*>
→ family inet → filter → input-list = "backbone-in"
(b) groups → INTERFACE-BACKBONE → interfaces → <ge-*> → unit <*>
→ family inet → filter → output = "interface-out"

Similar Configurations
(a) groups → INTERFACE-BACKBONE → interfaces → <ge-*> → unit <*>
→ family mpls → filter → input-list = "mpls-sample"

Referenceable Configurations
(a) groups → firewall → family inet → filter inet-sample → term default
→ then = sample
(b) groups → firewall → family inet → filter inet-sample → term default
→ then = next term
```

Figure 3: Example context mined on selected config line.

that are not immediately apparent, requiring a thorough and sometimes recursive analysis to trace how a value or policy is used across various parts of the configuration file.

Given the categorization of relevant contexts and the hierarchical tree model, we can automate the process of mining the aforementioned contexts by translating them into specific paths within the tree. This involves the following steps:

Neighboring Configuration Mining: Following the hierarchical tree structure of the configuration file, we can model the entire configuration as a tree T with nodes representing configuration elements and edges depicting the relationships between them. The root node V_1 represents the entry point of the configuration file and the V_2 nodes represent the broadest configuration categories. For a given configuration path $P = \{V_1 \to V_2(P) \to V_3(P) \to \ldots \to V_k(P) = v_k(P)\}$ within the tree, where $v_k(P)$ is the parameter value assigned to the parameter node $V_k(P)$, the goal of neighboring configuration mining is to extract other paths in the tree that share a common sub-path with P up to a certain depth.

The common ancestry level, or shared node depth, can be adjusted to control how much context is mined. Let us define m as the number of shared nodes from the root, then neighboring configurations are defined as:

$$N_m(P) = \{P' \in T \mid P' = V_1 \to V_2(P') \ldots \to V_m(P') \to V_{m+1}(P') \to \ldots \to V_k(P') = v_k(P')\},$$

where P' shares the first m nodes with P, but the nodes following $V_m(P')$ (denoted as $V_{m+1}(P'), V_{m+2}(P'), \ldots$) can differ from the remaining nodes in P, and P' does not end at the same parameter node as P, *i.e.*, $V_k(P') \neq V_k(P)$.

Adjusting m allows for control over the size and relevance of the neighboring context. A common choice is to set $V_m(P) = V_m(P') = V_{k-1}(P)$, which captures sufficient context while avoiding unnecessary complexity. For example. let the configuration path under review be represented as

$$P = \{V_1 \to A \to B \to C \to D = v_D\} \mid V_{k-1}(P) = C$$

The set of neighboring paths $N(P)$ can be defined as:

$$N(P) = \{P' \in T \mid P' = V_1 \to A \to B \to C \to D' = v_{D'}, D' \neq D\}$$

For example, consider a configuration line that defines the IP address for a particular interface:

$$P = \{V_1 \rightarrow \text{interfaces} \rightarrow \text{ge-0/0/0} \rightarrow \text{unit } 0 \rightarrow \text{family inet} \rightarrow \text{address} = 192.168.1.1/24\}$$

Here, $V_{k-1}(P)$ would be 'family inet'. Neighboring paths in this case might include:

$$N(P) = \{V_1 \rightarrow \text{interfaces} \rightarrow \text{ge-0/0/0} \rightarrow \text{unit } 0 \rightarrow \text{family inet} \rightarrow \text{mtu} = 1500\}$$

This ensures the extracted neighboring paths are closely relevant, revealing how related parameters are configured in the same section without overwhelming the system with excessive data. Thus, in our evaluation of CAIP, we select $V_{k-1}(P)$ as $V_m(P)$ for an effective balance between relevance and computational efficiency.

Similar Configuration Mining: To identify similar configurations, we focus on paths that share the same root node (V_1), broadest category node (V_2), and parameter node (V_k), but differ in their intermediate nodes or parameter values. Let $P = \{V_1 \rightarrow V_2(P) \rightarrow V_3(P) \rightarrow \ldots \rightarrow V_k(P) = v_k(P)\}$ represent the configuration line under review again. The set of similar configuration paths $S(P)$ is defined as:

$$S(P) = \{P' : P' = \{V_1 \rightarrow V_2(P') \rightarrow \ldots \rightarrow V_k(P') = v_k(P')\}\},$$

where P' shares $V_2(P') = V_2(P)$ and $V_k(P') = V_k(P)$, but may have different intermediate nodes $(V_3(P'), V_4(P'), \ldots)$ and a different parameter value $v_k(P')$.

By ensuring $V_2(P') = V_2(P)$, we compare configurations within the same category or context, even if the paths leading to $V_k(P')$ and $V_k(P)$ differ. Using only the root node V_1 would be too broad, as it encompasses the entire configuration file and does not provide meaningful differentiation. This approach allows us to understand how the same type of parameter is configured across different instances, which is crucial for ensuring consistency in network configurations.

For example, let the configuration path under review be:

$$P = \{V_1 \rightarrow \text{Interfaces} \rightarrow \text{Ethernet0} \rightarrow \text{IP} \rightarrow \text{MTU} = 1500\}$$

This represents the configuration for setting the Maximum Transmission Unit (MTU) to 1500 on interface Ethernet0. A similar configuration P' could be:

$$P' = \{V_1 \rightarrow \text{Interfaces} \rightarrow \text{Ethernet1} \rightarrow \text{IP} \rightarrow \text{MTU} = 9000\}$$

In this case, P' shares the same broad configuration category 'Interfaces' ($V_2(P') = V_2(P)$) and parameter node 'MTU' ($V_k(P') = V_k(P)$), but differs in the intermediate node 'Ethernet1' and the parameter value (9000 vs. 1500). By comparing similar configurations, the context reveals not only the MTU settings but also the intermediate configuration elements leading to the MTU setting across different interfaces within the network configuration.

Referenceable Configuration Mining: In the hierarchical tree structure, referenceable configurations are identified by locating paths where the parameter value of the current line under review appears as an intermediate node in other paths. This context is crucial for understanding how specific parameter values or policies are further referenced, elaborated upon, or applied in other parts of the configuration. we can formalize this set of referenceable configuration paths as:

$$R(P) = \{P' : P' = V_1 \rightarrow \ldots \rightarrow v_k(P) \rightarrow \ldots \rightarrow V_k(P')\} = v_k(P').$$

In these paths, $v_k(P)$ is no longer a parameter value, but an intermediate node, meaning it is being referenced or defined further down the configuration hierarchy.

For example, suppose the current configuration line is: $V_1 \rightarrow \mathit{RouterA} \rightarrow \mathit{Policy} \rightarrow \mathit{ImportPolicy} = \mathit{PolicyX}$, a referenceable path might be: $V_1 \rightarrow \mathit{RouterA} \rightarrow \mathit{Policy} \rightarrow \mathit{PolicyX} \rightarrow \mathit{Filter} = \mathit{AllowAll}$, indicating that *PolicyX* is further applied or modified by the filter configuration. Referenceable configuration mining helps to understand not only how a particular configuration value is defined, but also how it interacts with or influences other components of the network. This process is essential for detecting misconfigurations that arise from improper application or dependency handling across different sections of the configuration file.

One of the most important problems in extracting referenceable configurations is the risk of incorporating irrelevant context, particularly because the process does not distinguish between pre-defined parameter values inherent to the configuration language and custom values defined by network administrators. We now proceed to discuss this problem in more detail and present our solution.

3.1.2 Challenge 2 - Referenceable Parameter Value Ambiguity

In network configurations, parameter values have two types: *pre-defined* values and *user-defined* values. Pre-defined values are those that are built into the configuration language itself, such as boolean flags (True vs. False) or access control decisions (Allow vs. Deny). These values are understood by the configuration parser and typically do not require any additional definition or context within the configuration file. On the other hand, user-defined values are customizable by the network administrator and can vary widely depending on the specific requirements of the network. These include values such as IP addresses, timeout intervals, VLAN IDs, and other numeric or alphanumeric identifiers. However, there rarely exists documentation explicit specifying these types and requires a lot of manual examination, which is hard to scale.

When extracting context for referenceable configurations, failing to differentiate between pre-defined and user-defined

values can introduce irrelevant or misleading information into the extracted context. For instance, a pre-defined value like True is universally recognized by the configuration parser and could be used in multiple contexts, such as enabling a feature (FeatureX $\rightarrow$ Enabled = True) or setting a protocol flag (OSPF $\rightarrow$ PassiveInterface = True). Mining based on this pre-defined value could lead to irrelevant results, pulling in unrelated configurations that share the same boolean logic, thereby contaminating the context pool. Conversely, user-defined values like IP addresses, a prefix limit (*e.g.*, "maximum-prefixes 500" in a BGP configuration) or a timeout interval (*e.g.*, 30 seconds) are specific to the network and typically set by network operators. These values may appear in routing tables, NAT rules, or timeout policies, with their significance depending on how they're defined and applied. In this case, mining should focus on retrieving all related configurations that define or use these values.

Distinguishing between pre-defined and user-defined values is crucial to avoid extracting irrelevant context, which can lead to several issues:

1. *Context Contamination*: Irrelevant context introduced during the context mining process might erroneously group together configurations that pertain to entirely different aspects of network operation. This contamination of the context pool dilutes the relevance of the mined information, reducing the precision of the LLM and increasing the likelihood of false positives or missed errors.
2. *Reducing Computational Costs*: LLMs can be computationally expensive, especially when processing large-scale network configurations with many interdependent components. Distinguishing between pre and user-defined values can help optimizing the context extraction process, ensuring that only relevant and necessary contexts are passed to the model. This reduces the computational overhead associated with processing extraneous information.

Existence and Majority-Voting Based Differentiation: To accurately differentiate between pre-defined and user-defined parameter values within configuration files, we propose a solution that combines existence checks within the configuration tree and a majority-voting mechanism.

1. Existence-Based Differentiation: User-defined values often carry contextual information and are typically associated with further definitions or explanations elsewhere in the configuration file. For example, if a parameter value represents a specific, customized import policy, the configuration should contain other lines that elaborate on this policy's behavior. In the hierarchical tree model, if a parameter value is user-defined, it is likely to appear as an intermediate node in other configuration paths, indicating that it is referenced or elaborated upon elsewhere. Conversely, if no such paths exist, the value is likely pre-defined and requires no additional context. Consider the parameter RoutingPolicy with a value of ImportPolicy1 as an example. If ImportPolicy1 appears as an intermediate node in other configuration lines, such as those defining specific route maps or filters, it is likely user-defined. On the other hand, a parameter value like 'PassiveInterface = True' might not have any additional references, indicating that True is a pre-defined value.

Formal Definitions: Let Val_{user} be the set of user-defined values, and Val_{pre} be the set of pre-defined values. A parameter value v_k is considered *user-defined* if it appears as an intermediate node in at least one other configuration path $P' \in T$. This can be expressed as:

$$v_k \in \text{Val}_{\text{user}} \iff \exists P' = \{V_1 \rightarrow \ldots \rightarrow v_k \ldots \rightarrow V_k(P') = v_k(P')\} \in T$$

A parameter value v_k is considered *pre-defined* if it does not appear as an intermediate node in any other configuration path $P' \in T$. This can be formalized as:

$$v_k \in \text{Val}_{\text{pre}} \iff \nexists P' = \{V_1 \rightarrow \ldots \rightarrow v_k \ldots \rightarrow V_k(P') = v_k(P')\} \in T$$

2. Majority-Voting for Consistency: A shortcoming with existence-based differentiation is that the same parameter value can be user-defined in one context and pre-defined in another. For instance, the value 1000 used in a timeout setting might be pre-defined and require no further explanation. However, the same value 1000 used as a policy name or group identifier could be user-defined and require contextual elaboration. This distinction is crucial, as identical values can have different meanings depending on the associated configuration parameter. To address this ambiguity, we avoid assigning a universal type to a parameter value across all configurations. Instead, we determine whether a value is pre-defined or user-defined based on the specific combination of the configuration parameter and its associated value. For each configuration parameter, we analyze all associated values. If the majority of these values are user-defined, we classify the entire parameter (V_k) as well as all of its possible values (v_k) as user-defined. Conversely, if the majority are pre-defined, the parameter is classified accordingly. This approach leverages the principle that configuration parameters should exhibit uniformity in the type of values they accept, maintaining consistency across the configuration.

This further transforms our previous definition of Val_{user} and Val_{pre} into:

$$(V_k, v_k) = \begin{cases} \text{User-Defined}, \text{if } \sum_{i=1}^{n} \mathbb{1}\!\!\not\!\!\in (v_i \in \text{Val}_{\text{user}}) > \frac{n}{2}, \\ \text{Pre-Defined}, \text{if } \sum_{i=1}^{n} \mathbb{1}\!\!\not\!\!\in (v_i \in \text{Val}_{\text{pre}}) \geq \frac{n}{2}. \end{cases}$$

where n is the number of unique values associated with the configuration parameter V_k, and $\mathbb{1}\!\!\not\!\!\in$ is the indicator function that checks whether a value is user-defined or pre-defined.

Example: For the parameter Timeout, where values like 1000, 2000, and 3000 are used, if the majority lack associated

definitions, they are treated as pre-defined. Conversely, for a parameter like ImportPolicy, where values such as PolicyA, PolicyB, and 1000 (as a policy name) are used, if most have contextual definitions, all values under that parameter are treated as user-defined.

By combining existence-based checks with a majority-voting mechanism, we can effectively differentiate between pre-defined and user-defined parameter values in network configurations. This method ensures that the context extracted for LLM analysis is both relevant and accurate, thereby enhancing performance and reducing computational costs. Moreover, this approach maintains consistency within the configuration, ensuring that similar parameters are uniformly treated across different contexts.

3.2 Iterative Prompting

As we transition from context mining to utilizing the extracted information in LLM prompts, the next challenge is how to feed this information into the model without overwhelming it. While extracting accurate and relevant context is essential, the model's ability to process that information effectively is just as crucial.

3.2.1 Challenge 3 - Context Overload in Prompting

A significant limitation of many LLM-based approaches is context overload, where the model is fed too much information, causing it to lose focus on the most pertinent details. Even with the precise context extraction mechanisms we have, the extracted context can sometimes be extensive, leading to several issues:

1. *Dilution of Relevance*: The model might lose track of the most critical elements of the prompt, resulting in responses that are less accurate or less relevant. For instance, in a complex router configuration, key misconfiguration details could get buried in a flood of less relevant context.
2. *Loss of Coherence and Accuracy*: The model might produce disjointed responses or fail to address core misconfigurations when processing too much information at once. This can lead to fragmented reasoning or errors, especially with intricate router settings. Additionally, token limitations may cause the model to truncate important sections or struggle to process excessive context, further impacting performance.

For example, in detecting routing policy misconfigurations, a single misconfigured line may have dependencies across several sections of the file. Presenting all related context at once can cause the model to fail at prioritizing critical information, leading to missed/incorrect detection. STOA methods, such as prompt chaining, which guides the model through chained instructions, doesn't solve this issue, as it doesn't dynamically adjust or prioritize the context based on model needs.

Solution: To address these issues, CAIP implements an iterative, sequential prompting framework that allows the model to request and process relevant context in stages. Instead of overwhelming the model with all available context at once,

Initial Prompting

```
"I have a line of network route configuration for a Junipter router and
need to determine if there is any misconfiguration:
Configuration Line Under Review: groups-->INTERFACE-BACKBONE-->interfac
es--><xe-*>-->mtu=True
Given this information, please now proceed to analyze the provided con
figuration to identify any potential SYNTAX misconfigurations.
Do not hallucinate if no misconfiguration is occuring.
Additional contextual information with respect to the configuration li
ne under review can be provided upon request if useful for identifying
SYNTAX misconfigurations, include (1) Neighboring Configu
ration Lines; (2) Configuration Lines with Similar Pattern and Nature;
(3) Referenceable Configuration Lines; (4) Referenceable Configuration
Lines for Neigboring Configuration Line.
Respond in a json format strictly adhering to the following and do not
provide any other output:
If no further contextual information is needed:
{
    "Misconfigured": boolean, // true if there is any misconfiguration
    "Parameter": [], // List of misconfigured parameters
    "reason": [] // List of misconfiguration explanations
}
If further contextual information is needed:
{
    "requestedInfo": [], // List containing additionally requested
                         contextual information, select from [1,2,3,4]
    "reason": [] // List of explanations for requestedInfo
}"
```

Example LLM Response for Context Request

```
{
    "requestedInfo": [1, 2],
    "reason": ["Neighboring configuration lines are necessary to confirm
               the syntax structure in context.", "Configuration lines
               with a similar pattern and nature are needed to ensure
               consistent parameter usage."]
}
```

Figure 4: Example: initial prompting and LLM context request response for detecting syntax misconfiguration.

CAIP enables the model to actively request additional information as needed. This approach enhances the model's ability to maintain focus, coherence, and overall detection accuracy by allowing it to prioritize and process context more effectively, ensuring that only the most pertinent information is presented at each step.

1. Initial Prompting: The process begins by feeding the LLM the specific configuration line under review, along with targeted instructions about the type of misconfiguration we are checking for (*e.g.*, syntax errors, dependency conflicts, or just general misconfiguration). Figure 4 presents an example where the initial prompt includes MTU configuration line with an instruction to identify any syntax misconfigurations related to that configuration. This reduces the initial information load and allows the model to focus on the core issue.

2. Contextual Options: Next, CAIP offers the model the option to request additional context based on its understanding of the configuration line and the misconfiguration detection request. These options are based on our categorized context types, including: neighboring ($N(P)$), similar ($S(P)$) , and referenceable contexts ($R(P)$), as well as referenceable contexts on neighboring configurations ($N(R(P))$). By allowing the model to choose which context to receive, CAIP ensures that the LLM is only given information that is most relevant to the detection task at hand, reducing the risk of context overload. Figure 4 illustrates an example request response where the LLM requests neighboring context and referenceable contexts on neighboring configurations after the initial prompt to aid analyzing the MTU setting.

3. Iterative Refinement: After the model processes the initial prompt, it can request additional information if needed before

Additional Context Added to Prompt

```
Neighboring Configuration Lines:
 - groups-->INTERFACE-BACKBONE-->interfaces-->xe-*>-->unit <*>
    - family inet-->filter-->input-list-->1=backbone-in
    - family inet-->filter-->input-list-->0=uplink-in
    - family inet-->filter-->output=interface-out
    - family iso-->mtu=1497
    - family mpls-->filter-->input=mpls-sample
    - family inet6-->filter-->output=v6filter
    - family inet6-->mtu=9174
    - family inet6-->filter-->input-list-->0=inet6-sample
    - family inet6-->filter-->input-list-->1=v6filter
    - family mpls-->mtu=9174
    - family inet-->mtu=9174

Similar Configuration Lines:
 - groups
    - INTERFACE-BACKBONE-->interfaces-->ae*>-->unit <*>-->family iso-->
        mtu=1497
        (Further definition DO NOT EXIST for the value [1497] assigned
                to the parameter [mtu] in the rest of the configuration.)
    - INTERFACE-CONNECTOR-->interfaces-->ae*>-->mtu=9192
        (Further definition DO NOT EXIST for the value [9192] assigned
                to the parameter [mtu] in the rest of the configuration.)
```

Final LLM Response

```
{
    "Misconfigured": true,
    "Parameter": ["mtu"],
    "reason": ["The value 'True' assigned to the parameter 'mtu' is not
                valid. The 'mtu' parameter typically requires a numeric
                value representing the maximum transmission unit size."]
}
```

Figure 5: Example: Adding requested context and retrieving misconfiguration detection result.

arriving at the final decision. CAIP engages in a feedback loop where the model's output informs the next set of prompts. For instance, if the model identifies a possible syntax issue but requires more context from a related policy definition, *i.e.*, similar context, CAIP can deliver that specific context in the next prompt. This iterative process continues until the model has sufficient information to make a well-informed decision. Figure 5 provides an example of what a final misconfiguration decision looks like regarding a misconfiguared MTU value.

This sequential, interactive approach mirrors how a network administrator might manually review a configuration file—examining the most relevant sections first, then digging deeper into referenceable or adjacent configurations as needed.

3.2.2 Why Iterative, Requested Based Prompting Works

By breaking down the context into smaller, more digestible portions, CAIP addresses the fundamental challenges of context overload. This method significantly enhances the model's ability to:

1. Stay on target: With less irrelevant information, the model can maintain focus on the specific issue under review.
2. Preserve coherence: Incrementally adding information allows the model to build a more coherent understanding of the configuration [27, 44], improving detection of complex issues such as dependency conflicts or misapplied policies.
3. Optimize performance: Avoiding context overload reduces the processing burden on the model, leading to faster and more reliable outputs.

For example, in detecting misconfigurations in a multi-layer firewall policy, CAIP might first present the core rule in question, then iteratively offer neighboring rules that affect the same traffic flow, followed by referenceable configuration sections that define broader network policies. This structured process ensures that the LLM has all the information it needs, without drowning in unnecessary details, enabling it to make accurate decisions. Unlike conventional prompt-chaining, which provides incremental instructions based on a fixed context, CAIP dynamically adds context as requested by the LLM, focusing on the evolving needs of the analysis.

4 Evaluation

To evaluate the effectiveness of CAIP, we present two test scenarios as case studies and compare CAIP with state-of-the-art solutions. These include:

1. *Synthetic Misconfiguration Verification*: Synthetic misconfigurations are introduced into real network configuration snapshots. We run CAIP on specific configurations before and after the misconfigurations are introduced to evaluate the model's ability to accurately infer and detect misconfigurations.
2. *Comprehensive Real Configuration Snapshot Verification*: CAIP is run on configuration snapshots from a medium-scale campus network to detect overlooked misconfigurations in a real-world setting. This covers both *targeted* misconfiguration detection, where we focused on known issues like VLAN assignment errors, and *non-targeted* detection, which involved scanning for general misconfigurations without prior knowledge of the issues.

We now introduce the setup for our evaluation and present the details of the case studies in the subsequent sections.

4.1 Evaluation Setup

In this section, we describe our evaluation setup.

4.1.1 Context Extraction Hardware Setup

System Information: GNU/Linux Red Hat Enterprise Linux release 8.8 (Ootpa); *CPU Specifications*: Architecture: x86_64; CPU(s): 32 (2 threads/core, 16 cores/socket, 1 socket); Model: AMD EPYC 7302P 16-Core Processor; Max MHz: 3000.000; L3 cache: 16384K; NUMA node0 CPU(s): 0-31

We opt not to use the GPU, as the computational cost of the context extraction and processing tasks was manageable on the CPU — the tasks primarily benefit from parallel CPU threads, and GPU acceleration would not provide significant additional speedup.

4.1.2 LLM Model Used: GPT-4o

For running the LLM-based detection, we use GPT-4o [34], OpenAI's most recent transformer-based model designed for complex multi-step tasks.

Specifications: Model: GPT-4o-2024-05-13; Context window: 128,000 tokens; Max output tokens: 4,096 tokens; Training data Up to October 2023.

4.1.3 Prompting and Query Setup

We use OpenAI's Chat Completions API to interact with GPT-4o. The system was provided with a structured conversa-

Type	Description	Misconfig	Requested Context
Syntax	Missing brace	...interfaces: {"<ge-*":	None
	Invalid keyword	...mtu: "True"	$N(P)$, $R(P)$
	Incorrect hierarchy	...host-name: "{rtsw.alba-re1}"	$N(P)$
	Invalid IP address	...neighbor: 192.168.253.1.1"	$N(P)$, $R(P)$
Range	Invalid MTU	...mtu: "10000"	$N(P)$, $R(P)$
	Invalid VLAN ID	...vlan-id: "5000"	$N(P)$, $R(P)$, $N(R(P))$
	Invalid AS	...autonomous-system: 70000	$N(P)$, $R(P)$
	Invalid prefix limit	...maximum: "200000"	$N(P)$, $R(P)$
D/C	Non-existent group	... apply-groups: "L2-LSP-ATTRIBUTES"	$N(P)$, $R(P)$, $N(R(P))$
	Policy Conflict	...community add I2CLOUD-EXTENDED-TARGET	$N(P)$, $R(P)$, $N(R(P))$
	Non-existent filter	...input-list: "uplink"	$N(P)$, $R(P)$
	Non-existent policy	...export: "OESS-400-300-LOOP	$R(P)$
	Incorrect Filter Usage	...family mpls: filter:input-list:"v6filter"	$R(P)$,$S(P)$
	Disabled Sampling	...family inet6: SAMPLING MISSING/DISABLED	$R(P)$,$N(P)$
	VRF Target Conflict	...vrf-target target:11537:"313001"	$S(P)$
	Abnormal Small MTU	...family iso: mtu: "1497"	$R(P)$,$S(P)$

Table 1: Synthetic Misconfigurations Introduced and LLM Requested Context for CAIP.

tion history to maintain context across multiple queries. For each query, the model was tasked with analyzing the configuration lines to detect potential misconfigurations, with instructions tailored to focus on specific misconfiguration types (*e.g.*, syntax issues, policy conflicts) or general misconfigurations. Figures 4 and 5 show example prompting, queries, and responses.

4.2 Case Study 1: Synthetic Misconfiguration Verification

We first evaluate CAIP on synthetically introduced misconfigurations, which allows us to systematically test various misconfiguration types and assess detection accuracy in a controlled environment. We broadly categorize router misconfigurations into three types:

- *Syntax errors* occur when the configuration does not adhere to the expected format or structure—e.g., a missing bracket or misused keyword in a BGP routing policy.
- *Range violations* involve parameter values that fall outside the acceptable range—e.g., an MTU value that exceeds the maximum allowed for a specific interface type.
- *Dependency/conflict (D/C) issues* arise when different configuration lines are incompatible or contradict each other—e.g., a firewall rule might block traffic that another policy explicitly permits.

To evaluate CAIP, we obtain snapshot configurations from Internet2 (Juniper devices) and sampled 16 correct configuration lines. These were then modified to introduce synthetic errors across the three misconfiguration types. For syntax and range errors, we created four distinct misconfigurations each, totaling 8 errors (see Table 1). For D/C misconfigurations, we introduced 8 distinct errors, as vanilla LLMs excel at detecting syntax and range issues, while CAIP's context mining is particularly effective at uncovering nuanced D/C issues. The additional D/C errors allow us to thoroughly assess this capability. For each misconfiguration, we run CAIP by following the two components: (1) treating the misconfigured line as the line under review and extracting all relevant context, and (2) conducting the iterative, sequential prompting process against the model, obtaining the final misconfiguration detection decision.

When forming the prompt, we explicitly instruct the model to look for each type of misconfiguration—syntax, range, or dependency/conflict—individually. This is because the model may request different types of context depending on the specific misconfiguration type it is trying to detect. We report only the results corresponding to the actual misconfiguration type introduced. Importantly, CAIP never yielded false positives when the model was instructed to find a misconfiguration type different from the actual type. Additionally, we find that asking the model to search for 'GENERAL' misconfigurations also led to successful detection, though more context was often requested by the model.

We compare CAIP to three representative tools: the model checker *Batfish* [14], the consistency checker *Diffy* [20], and the partition-based GPT Q&A model *Ciri* [29] using GPT-4o for fair comparison.

Type	Cases	Number of Corrected Detected Cases			
		CAIP	Ciri	Batfish	Diffy
Syntax Error	4	4/4 (100%)	2/4 (50%)	3/4 (75%)	0/4 (0%)
Range Error	4	4/4 (100%)	2/4 (50%)	0/4 (0%)	0/4 (0%)
D/C Error	8	8/8 (100%)	1/8 (12.5%)	2/8 (25%)	0/8 (0%)
Original Configs (No Error)	16	16/16 (100%)	16/16 (100%)	16/16 (100%)	16/16 (100%)
Total	**32**	**32/32 (100%)**	**21/32 (65.6%)**	**21/32 (65.6%)**	**16/32 (50%)**

Table 2: Synthetic Misconfiguration Detection Results.

The results in Table 2 demonstrate that CAIP outperforms the other tools in detecting all three types of misconfigurations. It achieves a perfect 16/16 detection rate across the board for misconfigurations. This indicates that CAIP's comprehensive context mining and iterative prompting process effectively identify errors in network configurations, even in more complex cases like D/C issues. In contrast, the other tools showed limitations, particularly when it came to detecting range violations and D/C issues.

Comparison Against Partition-based LLMs The partition-based GPT model Ciri performs reasonably well in detecting syntax and range violations, with a detection rate of 50% for both categories. This success is due to the LLM model's training on vast, diverse text, including network-related documentation and configuration files, enabling it to recognize common syntax structures and numerical limits typically found in configurations. Transformer-based LLMs, with their self-attention mechanism, are particularly effective at identifying these common, localized issues. However, it lags significantly behind CAIP in detecting D/C issues, with a 12.5% detection rate in this category. The primary reason for this underperformance is the partitioning approach. This approach analyzes configuration sections in isolation, failing to capture the complex interdependencies between configuration lines, which is crucial for detecting D/C issues. The only D/C misconfiguration that Ciri successfully detects is the 'VRF Target Conflict,' where the conflicting 'route-distinguisher' value configuration happens to be in the same prompt partition, providing just enough context for accurate detection.

As shown in Table 1 and 3, D/C issues, such as policy conflicts, require the model to understand and analyze in-

teractions between different configuration elements. CAIP addresses this by guiding the LLM to request the relevant contextual information it needs for accurate reasoning and detection. The example CAIP-enable LLM responses in Table 3 clearly show that these contexts are being leveraged to derive the correct detection results. In contrast, partition-based GPT struggles to detect such misconfigurations effectively, as it analyzes configurations in isolation and can often miss these complex interdependencies.

Type	Misconfig	errParameter	Reason
Syntax	Missing brace	["<ge-*>"]	"MTU value for 'family inet' is set to 9174 which is less than the interface MTU value of 9192. The MTU for 'family inet' should not be less than the interface MTU."
Range	Invalid VLAN ID	["maximum"]	"The 'maximum' value of 200000 in the configuration line under review significantly exceeds the 'maximum' values of 1000 found in similar configuration lines, indicating a potential range misconfiguration."
D/C	Policy Conflict	["remove-comms-rt"]	"There is a conflict in the configuration under term 'remove-comms-rt' where the action 'community add I2CLOUD-EXTENDED-TARGET' is followed by 'community delete I2CLOUD-EXTENDED-TARGET' which are contradictory actions."
	Disabled Sampling	["inet6"]	"The family mpls is incorrectly using the IPv6 filter inet6-sample v6filter, which is intended for family inet6 in similar configuration lines. MPLS traffic should have its own specific filter, as the filtering requirements for MPLS are different from those for IPv6."
	VRF Target Conflict	["target"]	"The route-distinguisher (RD) in neighoring configurations is set to 11537:381841, while the vrf-target is set to target:11537:313001. These values should typically align, or at least follow certain conventions for mapping VRFs to the correct target instances in MPLS/BGP L2VPN."
	Incorrect Filter Usage	["input-list"]	"The family mpls is incorrectly using the IPv6 filter inet6-sample v6filter, which is intended for family inet6 in similar configuration lines. MPLS traffic should have its own specific filter, as the filtering requirements for MPLS are different from those for IPv6."

Table 3: Example LLM Reasoning for CAIP.

Comparison Against Non-LLM-based Tools Batfish performs well on syntax errors (75% detection rate) but struggles with range violations and only partially detected D/C issues (2/8). Batfish uses a rule-based model checker, which excels in detecting predefined syntax errors but is less effective in handling misconfigurations that involve contextual nuances, such as range violations or dependency conflicts. Specifically, Batfish detects issues related to non-existent groups and policies (hard-coded into its rule checks) but missed more subtle D/C misconfigurations, like policy conflicts or filter dependencies.

Diffy performs poorly across all misconfigurations, with a 0/16 detection rate, due to its data-driven approach, which focuses on learning common usage patterns from configurations and flagging anomalies as potential bugs. While effective for detecting clear deviations from standard patterns in other configuration files, complex issues like policy conflicts or range violations may not deviate from typical patterns but still be incorrect, which Diffy's template-based anomaly detection cannot capture. Furthermore, Diffy's reliance on identifying deviations from learned patterns makes it less effective at detecting syntax errors and range violations. These types of misconfigurations often follow standard formatting and numerical values that may not stand out as anomalies, making them difficult for Diffy to detect without a granular understanding of configuration structure and constraints.

Finally, despite varying performance in detecting misconfigurations, all tools, including CAIP, Batfish, Diffy, and the partition-based GPT model Ciri, successfully marked all correctly configured lines as valid prior to the introduction of the synthetic errors, indicating strong capabilities in avoiding false positives for correctly functioning configurations.

4.3 Case Study 2: Comprehensive Real Configuration Snapshot Verification

To evaluate CAIP on real-world misconfiguration detection, we obtain Aruba router configuration snapshots from a medium-scale campus network. We have two objectives: (1) to verify the scalability of CAIP when applied to larger network configurations, and (2) to investigate whether CAIP can detect potential misconfigurations that have not yet been identified by existing tools.

We perform two types of analysis, beginning with *targeted misconfiguration detection:* A key aspect of this case study is the demonstration of CAIP's flexibility in integrating additional context types under different scenarios. This is particularly useful when network operators have prior domain knowledge about specific misconfigurations they are trying to detect that fall outside of CAIP's default context library. To illustrate this flexibility, we consult the network operators who provided the configuration snapshots. They were specifically interested in detecting incorrect 'VLAN assignments' — a type of misconfiguration that often requires a network-wide view. Such misconfigurations are usually identifiable only when considering the configurations of other devices in the same network, as deviations from uniform configurations can indicate potential issues. To address this, we introduce an additional, default context type, called 'Intra-Router Consistency Context'. This context type mines the prevalence of the same parameter-value pair across other devices in the network, providing insights into whether a configuration is common or potentially erroneous. For this scenario, we modify the initial prompt to focus specifically on detecting 'WRONG VLAN ASSIGNMENT'.

Example Intra-Router Consistency Context extracted:

'For the Configuration Line Under Review, the same configuration is found in 189 out of 191 other configuration files. (Significantly lower prevalence may indicate an uncommon or potentially erroneous configuration.)'

The second analysis involves *non-targeted misconfiguration detection*, where CAIP uses its default context library to aim the LLM for detecting general misconfigurations without prior knowledge of specific issues. In this mode, no specific misconfiguration type is provided to the model; instead, CAIP exhaustively analyzes the entire configuration file to identify 'GENERAL' errors. This approach is particularly valuable for detecting overlooked misconfigurations in large, complex network environments where errors may not be immediately obvious or anticipated. By running CAIP in this mode, we assess its ability to detect subtle misconfigurations across the network without predefined expectations, ensuring it can operate effectively even when network operators are unsure of

Analysis Type	Misconfigs	Count	Severity (Reason)	True Positive Rate (TPR)
Targeted	Wrong VLAN Assignment	6	*High*: Wrong VLANs permitted in configuration, causing access issues.	2/6 (33.3%)
Non-Targeted	Invalid Subnet Mask	2	*High*: Subnet mask usage, *e.g.*, 255.255.253.0) violating binary boundary rules.	2/2 (100%)
	Inconsistent Policy Naming	14	*Low*: Misnamed 'smnpread' for 'snmp _communities',but with consistent usage.	14/14 (100%)
	Insecure SSH KEX Algorithm	1	*Low*: Insecure 'diffie-hellman-group14-sha1' used (low risk as stronger algorithms are included).	1/1 (100%)
	Ambiguous Rate Limit Values	2	*Ambiguous*: 'Interval' and 'burst' set to 0, leading to nondeterministic behaviors.	2/2 (100%)

Table 4: Comprehensive Real Configuration Snapshot Verification Results: CAIP-Detected Misconfigurations

the specific issues they might face.

We perform exhaustive analysis using CAIP across 11 configuration files covering $\sim 6\%$ of all devices on the campus network, applying the context mining framework to extract all relevant context for each configuration line. We then prompt the model to identify the corresponding misconfigurations, allowing it to dynamically request the necessary context during the iterative prompting phase. The detection results are presented in Table 4, highlighting CAIP's ability to uncover new misconfigurations. For each of misconfigurations detected by CAIP, we verify the correctness of the inference decision with domain experts.

Targeted Analysis In the targeted analysis, CAIP demonstrates consistent performance in identifying misconfigurations related to incorrect VLAN assignments. Out of the six misconfigurations detected, CAIP correctly flags two with a true positive rate (TPR) of 33.3%. The identified misconfigurations involved wrong VLAN assignments that caused misrouting and access issues, a high-severity problem for network operations.

Despite the modest TPR, the false positives align with the domain experts' expectations. Network experts clarified the VLAN assignment deviations falsely flagged as misconfigurations are in fact intentional modifications tailored to the specific routers within the network. In this regard, we compare CAIP's detection results (both true and false positives) with those from the experts' internal tool—a graph-based, non-LLM solution designed to check intra-router consistency—and the results are consistent. Thus, while the false positives may appear significant, they represent deviations from standard configurations that would typically signal errors, but in these cases, are expected and intentional adjustments.

Non-Targeted Analysis In the non-targeted analysis, CAIP was able to detect several types of misconfigurations across the provided configuration snapshots, as shown in Table 4. The detected misconfigurations were categorized by severity based on their potential impact on network operations:

- *High Severity*: Misconfigurations in this category pose critical risks to the network and require immediate action — CAIP identified invalid subnet mask (e.g., 255.255.253.0) usages that violate binary boundary rules, which could lead to routing issues and network instability.
- *Low Severity*: These issues may not immediately disrupt network operations but should be addressed to ensure optimal functionality and avoid potential risks— CAIP identified many instances of misnamed configurations (*e.g.*, 'smnpread' instead of 'snmpread' for 'snmp_communities') and inclusion of insecure SSH KEX algorithm which ('diffie-hellman-group14-sha1'), do not break the system but could lead to confusion and maintenance challenges.
- *Ambiguous*: These misconfigurations are related to configurations that lead to undefined or nondeterministic behaviors — CAIP found ambiguous rate limit values ('interval' and 'burst' set to 0), which can cause unpredictable performance issues due to missing documentation on how these values should be handled, suggesting updates or further specifications in the documentation are needed.

Specifically, CAIP successfully identified 19 misconfigurations, including two cases of invalid subnet masks, 14 instances of inconsistent policy naming, one insecure SSH Key Exchange (KEX) algorithm, and two ambiguous rate limit values. Validated with domain experts, we find all misconfiguration cases are either valid or justifiable with a TPR of 100% across all targeted misconfigurations.

Across synthetic and real-world configurations, CAIP consistently outperforms state-of-the-art tools like Batfish, Diffy, and partition-based GPT (Ciri), especially in detecting complex misconfigurations. CAIP's dynamic context mining and iterative prompting allow it to capture interdependencies that other methods missed. Both in targeted and non-targeted scenarios, it demonstrates capability in identifying misconfigurations that traditional tools overlooked, making it a robust solution for network configuration validation.

5 Discussion and Future Work

CAIP has proven effective in improving the detection of router misconfigurations, but there are several avenues for further exploration and refinement. One challenge is expanding the framework to better handle large-scale, distributed networks with interdependent routers. These environments often involve complex relationships between devices, and future work could focus on extending CAIP to capture and analyze multi-device contexts by default. Incorporating cross-router consistency checks and advanced network-wide policy validation could be valuable enhancements.

Another area for improvement is enhancing the LLM's ability to process real-time configuration changes. As networks evolve dynamically, incorporating real-time monitoring data into the context mining process would allow for continuous verification and faster misconfiguration detection. Integrating CAIP with network management tools that track operational metrics could provide valuable runtime data, further improving detection accuracy by correlating live data with configuration snapshots.

Future enhancements can also involve proposing and implementing automated fixes for detected misconfigurations with real-time updates. This would help network operators quickly address issues, streamlining troubleshooting and repairs.

6 Conclusion

In this paper, we introduced CAIP, a framework for router misconfiguration detection that leverages context-aware iterative prompting to improve LLM accuracy. By systematically extracting relevant context from network configurations and following a guided, interactive prompting mechanism, CAIP addresses limitations in current model checkers, consistency checkers, and LLM-based models. Through case studies involving both synthetic and real-world misconfigurations, we demonstrated that CAIP outperforms existing methods across different misconfiguration types, ensuring more reliable and scalable network configuration verification.

References

[1] Anubhavnidhi Abhashkumar, Aaron Gember-Jacobson, and Aditya Akella. Tiramisu: Fast multilayer network verification. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 201–219, 2020.

[2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[3] Ehab Al-Shaer and Mohammed Noraden Alsaleh. Configchecker: A tool for comprehensive security configuration analytics. In *2011 4th Symposium on Configuration Analytics and Automation (SAFECONFIG)*, pages 1–2. IEEE, 2011.

[4] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. A general approach to network configuration verification. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 155–168, 2017.

[5] Theophilus Benson, Aditya Akella, and David Maltz. Unraveling the complexity of network management. In *Symposium on Networked Systems Design and Implementation (NSDI)*, 2009.

[6] Rudiger Birkner, Tobias Brodmann, Petar Tsankov, Laurent Vanbever, and Martin Vechev. Metha: Network verifiers need to be correct too! In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2021.

[7] Mark Bogdanov. *Leveraging Advanced Large Language Models To Optimize Network Device Configuration*. PhD thesis, Purdue University Graduate School, 2024.

[8] Tom B Brown. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[9] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020.

[10] Xie Chen, Yu Wu, Zhenghao Wang, Shujie Liu, and Jinyu Li. Developing real-time streaming transformer transducer for speech recognition on large-scale dataset. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5904–5908. IEEE, 2021.

[11] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 674–688, 2024.

[12] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

[13] Nick Feamster and Hari Balakrishnan. Detecting bgp configuration faults with static analysis. In *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*, pages 43–56, 2005.

[14] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. A general approach to network configuration analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 469–483, 2015.

[15] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[16] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.

[17] Felix Hill. Why transformers are obviously good models of language. *arXiv preprint arXiv:2408.03855*, 2024.

[18] Alan Jeffrey and Taghrid Samak. Model checking firewall policy configurations. In *2009 IEEE international symposium on policies for distributed systems and networks*, pages 60–67. IEEE, 2009.

[19] Siva Kesava Reddy Kakarla, Alan Tang, Ryan Beckett, Karthick Jayaraman, Todd Millstein, Yuval Tamir, and George Varghese. Finding network misconfigurations by automatic template inference. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 999–1013, 2020.

[20] Siva Kesava Reddy Kakarla, Francis Y Yan, and Ryan Beckett. Diffy: Data-driven bug finding for configurations. *Proceedings of the ACM on Programming Languages*, 8(PLDI):199–222, 2024.

[21] Anjali Khurana, Hariharan Subramonyam, and Parmit K Chilana. Why and when llm-based assistants can go wrong: Investigating the effectiveness of prompt-based interactions for software help-seeking. In *Proceedings of the 29th International Conference on Intelligent User Interfaces*, pages 288–303, 2024.

[22] Franck Le, Sihyung Lee, Tina Wong, Hyong S Kim, and Darrell Newcomb. Characterization and problem detection of routing policy configurations. 2006.

[23] Franck Le, Sihyung Lee, Tina Wong, Hyong S Kim, and Darrell Newcomb. Minerals: using data mining to detect router misconfigurations. In *Proceedings of the 2006 SIGCOMM workshop on Mining network data*, pages 293–298, 2006.

[24] Franck Le, Sihyung Lee, Tina Wong, Hyong S Kim, and Darrell Newcomb. Detecting network-wide and router-specific misconfigurations through data mining. *IEEE/ACM transactions on networking*, 17(1):66–79, 2008.

[25] Franck Le, Geoffrey G. Xie, and Hui Zhang. Understanding route redistribution. In *2007 IEEE International Conference on Network Protocols (ICNP)*, 2007.

[26] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. Can long-context large language models understand long contexts?

[27] Lei Li, Yongfeng Zhang, and Li Chen. Prompt distillation for efficient llm-based recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 1348–1357, 2023.

[28] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhu Chen. Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060*, 2024.

[29] Xinyu Lian, Yinfang Chen, Runxiang Cheng, Jie Huang, Parth Thakkar, and Tianyin Xu. Configuration validation with large language models. *arXiv preprint arXiv:2310.09690*, 2023.

[30] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers. *AI open*, 3:111–132, 2022.

[31] Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, and Shane Luke. Prompt compression with context-aware sentence encoding for fast and improved llm inference. *arXiv preprint arXiv:2409.01227*, 2024.

[32] Chang Liu, Xiaohui Xie, Xinggong Zhang, and Yong Cui. Large language models for networking: Workflow, advances and challenges. *arXiv preprint arXiv:2404.12901*, 2024.

[33] Houlsby Neil and Weissenborn Dirk. Transformers for image recognition at scale. *Online: https://ai. googleblog. com/2020/12/transformers-for-image-recognitionat. html*, 2020.

[34] OpenAI. Hello gpt-4o. https://openai.com/index/hello-gpt-4o/, 2024. Accessed: 2024-09-17.

[35] Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Noam Shazeer, Alexander Ku, and Dustin Tran. Image transformer. In *International conference on machine learning*, pages 4055–4064. PMLR, 2018.

[36] Santhosh Prabhu, Kuan Yen Chou, Ali Kheradmand, Brighten Godfrey, and Matthew Caesar. Plankton: Scalable network configuration verification through model checking. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 953–967, 2020.

[37] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Yujia Zhou, Xu Chen, and Zhicheng Dou. Are long-llms a necessity for long-context tasks? *arXiv preprint arXiv:2405.15318*, 2024.

[38] Xipeng Qiu, Tianxiang Sun, Yige Xu, Yunfan Shao, Ning Dai, and Xuanjing Huang. Pre-trained models for natural language processing: A survey. *Science China technological sciences*, 63(10):1872–1897, 2020.

[39] Ronald W Ritchey and Paul Ammann. Using model checking to analyze network vulnerabilities. In *Proceeding 2000 IEEE Symposium on Security and Privacy. S&P 2000*, pages 156–165. IEEE, 2000.

[40] Murray Shanahan. Talking about large language models. *Communications of the ACM*, 67(2):68–79, 2024.

[41] Fenfen Sheng, Zhineng Chen, and Bo Xu. Nrtr: A no-recurrence sequence-to-sequence model for scene text recognition. In *2019 International conference on document analysis and recognition (ICDAR)*, pages 781–786. IEEE, 2019.

[42] Yan Shvartzshnaider, Vasisht Duddu, and John Lacalamita. Llm-ci: Assessing contextual integrity norms in language models. *arXiv preprint arXiv:2409.03735*, 2024.

[43] Samuel Steffen, Timon Gehr, Petar Tsankov, Laurent Vanbever, and Martin T. Vechev. Probabilistic verification of network configurations. In *SIGCOMM*, 2020.

[44] Hari Subramonyam, Roy Pea, Christopher Pondoc, Maneesh Agrawala, and Colleen Seifert. Bridging the gulf of envisioning: Cognitive challenges in prompt based interactions with llms. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pages 1–19, 2024.

[45] Alan Tang, Siva Kesava Reddy Kakarla, Ryan Beckett, Ennan Zhai, Matt Brown, Todd D. Millstein, Yuval Tamir, and George Varghese. Campion: debugging router configuration differences. In *ACM SIGCOMM 2021 Conference*, 2021.

[46] Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. Galactica: A large language model for science. arxiv 2022. *arXiv preprint arXiv:2211.09085*, 10, 2023.

[47] Xiaoyi Tian, Amogh Mannekote, Carly E Solomon, Yukyeong Song, Christine Fry Wise, Tom Mcklin, Joanne Barrett, Kristy Elizabeth Boyer, and Maya Israel. Examining llm prompting strategies for automatic evaluation of learner-created computational artifacts. In *Proceedings of the 17th International Conference on Educational Data Mining*, pages 698–706, 2024.

[48] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[49] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[50] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, and Marco Chiesa. Netconfeval: Can llms facilitate network configuration? *Proceedings of the ACM on Networking*, 2(CoNEXT2):1–25, 2024.

[51] Zehao Wang, Dong Jae Kim, and Tse-Hsun Chen. Identifying performance-sensitive configurations in software systems through code analysis with llm agents. *arXiv preprint arXiv:2406.12806*, 2024.

[52] Xieyang Xu, Weixin Deng, Ryan Beckett, Ratul Mahajan, and David Walker. Test coverage for network configurations. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.

[53] Fuzhao Xue, Yao Fu, Wangchunshu Zhou, Zangwei Zheng, and Yang You. To repeat or not to repeat: Insights from scaling llm under token-crisis. *Advances in Neural Information Processing Systems*, 36, 2024.

[54] Fangdan Ye, Da Yu, Ennan Zhai, Hongqiang Harry Liu, Bingchuan Tian, Qiaobo Ye, Chunsheng Wang, Xin Wu, Tianchen Guo, Cheng Jin, Duncheng She, Qing Ma, Biao Cheng, Hui Xu, Ming Zhang, Zhiliang Wang, and Rodrigo Fonseca. Accuracy, scalability, coverage: A practical configuration verifier on a global WAN. In *SIGCOMM*, 2020.

[55] Yao-Ching Yu, Chun-Chih Kuo, Ziqi Ye, Yu-Cheng Chang, and Yueh-Se Li. Breaking the ceiling of the llm community by treating token generation as a classification for ensembling. *arXiv preprint arXiv:2406.12585*, 2024.

[56] Hongyi Zeng, Peyman Kazemian, George Varghese, and Nick McKeown. Automatic test packet generation. In *CoNEXT*, 2012.

[57] Peng Zhang, Dan Wang, and Aaron Gember-Jacobson. Symbolic router execution. In Fernando Kuipers and Ariel Orda, editors, *ACM SIGCOMM Conference*, 2022.